

VISIÓN GENERAL DEL SET DE INSTRUCCIONES RISC-V

Los registros son variables de acceso rápido dentro de la CPU y la memoria es donde se guardan los datos de forma permanente.

Ejemplo de instrucción:

```
1 addi x1, x0, 5
```

Tras ejecutarlo:

```
1 x1 = 5
```

Para facilitar la programación, también existen pseudo instrucciones que el ensamblador convierte en varias instrucciones RISC-V:

```
1 li t0, 5
2 li t1, 7
3 add t2, t0, t1
```

Que se ejecuta como:

```
1 addi t0, x0, 5 # (o, lo que es lo mismo, addi t0, zero, 5)
2 addi t1, x0, 7
3 add t2, t0, t1
```

Tras ejecutarlo:

```
1 t0 = 5
2 t1 = 7
3 t2 = 12
```

TIPOS DE INSTRUCCIONES

En RISC-V las instrucciones pertenecen a uno de estos formatos:

- **R**: operaciones entre registros.
- **I**: operaciones inmediatas y acceso a memoria (carga).
- **S**: almacenamiento en memoria.
- **B**: saltos condicionales.
- **U**: carga de valores grandes.
- **J**: saltos incondicionales.

1. Tipo R (Register)

Realizan operaciones utilizando únicamente registros.

Es el formato típico para:

- Sumas
- Restas
- Operaciones lógicas
- Comparaciones

Instrucciones típicas

add

Que es una de las que hemos visto en el ejemplo inicial. La otra **addi** es de tipo **I**. Por último, **li** no es una instrucción sino una seudo instrucción.

Suma dos registros. Vemos otro ejemplo como el inicial.

```
1  add t0, t1, t2
```

Equivale a:

```
1  t0 = t1 + t2;
```

Ejemplo:

```
1  li t1, 10
2  li t2, 20
3  add t0, t1, t2
```

Resultado:

```
1  t0 = 30
```

sub

Resta dos registros.

```
1 sub t0, t1, t2
```

Equivale a:

```
1 t0 = t1 - t2;
```

and

AND lógico bit a bit.

```
1 and t0, t1, t2
```

Ejemplo:

```
1 t1 = 0b1100 # en hexadecimal 0xc, en decimal 12
2 t2 = 0b1010 # en hexadecimal 0xa, en decimal 10
```

Resultado:

```
1 t0 = 0b1000 # en hexadecimal y en decimal 8
```

En RARS no podemos utilizar 0b... Hay que convertir el número a hexadecimal o binario.

or

OR lógico.

```
1 or t0, t1, t2
```

Ejercicios

Ejercicio 1

Calcular:

```
1 25 + 17
```

y guardar el resultado en t2.

Ejercicio 2

Calcular:

```
1 100 - 35
```

y guardar el resultado en t0. Repetir el ejercicio con otra instrucción final distinta.

Ejercicio 3

Realizar un OR lógico entre:

1	1100
2	0011

2. Tipo I (Immediate)

Permiten trabajar con:

- Constantes
 - Cargas desde memoria
 - Desplazamientos
-

Instrucciones típicas

addi

Suma una constante.

1	addi t0, t1, 10
---	-----------------

Equivale a:

1	t0 = t1 + 10;
---	---------------

Ejemplo:

1	li t1, 25
2	addi t0, t1, 5

Resultado:

1	t0 = 30
---	---------

andi

AND con una constante.

1	andi t0, t1, 15
---	-----------------

lw

Carga una palabra de memoria.

```
1 lw t0, 0(s0)
```

Significa:

```
1 t0 = memoria[s0];
```

Si:

```
1 s0 = 10010000
2 memoria[10010000] = 42
```

entonces:

```
1 t0 = 42
```

Ejercicios

Ejercicio 1

Incrementar el contenido de t0 en 1.

Ejercicio 2

Sumar 50 al registro s1.

Ejercicio 3

Cargar en t1 el contenido almacenado en la dirección apuntada por la dirección que se obtiene de sumar 8 bytes a la indicada en s2, antes guarda en S2 el valor 0x10010024 y en la dirección de memoria 0x1001002c el valor 0x20554158. Si quieres, puedes seleccionar la casilla ASCII y ver qué hay en la dirección de donde ha salido el dato para averiguar qué significa lo que acabas de cargar en t1.

3. Tipo S (Store)

Guardan datos desde registros hacia memoria.

Recordamos que lw , que es de tipo I, lee memoria.

Ahora veremos sw escribe memoria.

Instrucciones típicas

sw

Store Word.

```
1 sw t0, 0(s0)
```

Equivale a:

```
1 memoria[s0] = t0;
```

Ejemplo:

```
1 li t0, 123
2 sw t0, 0(s0)
```

Tras ejecutarse:

```
1 memoria[s0] = 123
```

sb

Store Byte.

Guarda un byte.

Recuerda que sw guarda una palabra (4 bytes; 8 bits x 4 bytes = 32 bits)

```
1 sb t0, 0(s0)
```

sh

Store Halfword.

Guarda 16 bits (media palabra).

```
1 sh t0, 0(s0)
```

Ejercicios

Ejercicio 1

Guardar el valor de t0 en la dirección apuntada por s0. Antes carga en t0 el valor 0xffffffff y en s0 el valor 0x10010080. Repítelo ahora cargado sólo el byte menos significativo.

Ejercicio 2

Repite el ejercicio 1, pero ahora guardando sólo la mitad menos significativa de lo que contiene t0.

Ejercicio 3

Repite el ejercicio 1, pero ahora guardando sólo el byte menos significativo de lo que contiene t0.

Ejercicio 4

Guardar el número 55 en memoria. Elige en qué registro guardar inicialmente el valor y en qué localidad de memoria lo quieres guardar.

4. Tipo B (Branch)

Realizan saltos condicionales.

Instrucciones típicas

beq

Branch if equal.

```
1 beq t0, t1, etiqueta
```

Si:

```
1 t0 == t1
```

salta.

Ejemplo:

```
1 li t0, 5
2 li t1, 0
3 continuar:
4 addi t1, t1, 1
5 beq t0, t1, fin
```

6	j continuar
7	fin:j fin

bne

Branch if not equal.

1	bne t0, t1, etiqueta
---	----------------------

blt

Branch if less than.

1	blt t0, t1, menor
---	-------------------

bge

Branch if greater or equal.

1	bge t0, t1, mayor
---	-------------------

Ejercicio

Escribir un bucle que cuente de 0 a 9.

Pista:

1	addi
2	blt

5. Tipo U (Upper Immediate)

Sirve para construir constantes grandes porque las instrucciones normales no pueden contener valores arbitrariamente grandes.

Instrucciones típicas

lui

Load Upper Immediate.

```
1 lui t0, 0x12345
```

Carga:

```
1 0x12345000
```

en t0.

auipc

Add Upper Immediate to PC.

Se utiliza mucho para:

- Direcciones
- Código independiente de posición
- Acceso a variables globales

```
1 auipc t0, 0
```

Ejercicios

Ejercicio 1

Construir el valor:

```
1 0x12345000
```

utilizando lui.

Ejercicio 2

Investigar qué valor deja en el registro:

```
1 lui t0, 1
```

6. Tipo J (Jump)

Permiten realizar saltos incondicionales.

Instrucciones típicas

jal

Jump and Link.

Salta y guarda la dirección de retorno en ra (x1).

```
1 jal ra, funcion
```

Equivale aproximadamente a una llamada a función.

$Ra = PC + 4$

PC = función

Ejemplo:

```
1 jal ra, suma
2
3 ...
4
5 suma:
6     ret
```

ret, para regresar a la dirección de retorno, equivale a:

jalr x0, 0(ra)

j (pseudoinstrucción)

```
1 j bucle
```

El ensamblador la convierte internamente en una forma de jal.
